UNIVERSITY OF MARIE AND LOUIS PASTEUR

GPS DISCIPLINED QUARTZ OSCILLATOR

GPSDO

ATCE PROJECT

presented by :

HEROUMI Yani
NEGRECHE Aaron
ATMANI Aymene
CHAOUCHE Mastene
BELAIDI Katia
ATAMNIA Mondher

supervised by :
Pr. Jean Michel Friedt
Pr. Emile Carry

Academic : 2025/2026

WHY DO WE NEED A GPSDO ?

Quartz is widely used in electronics because it is accurate, small, and inexpensive

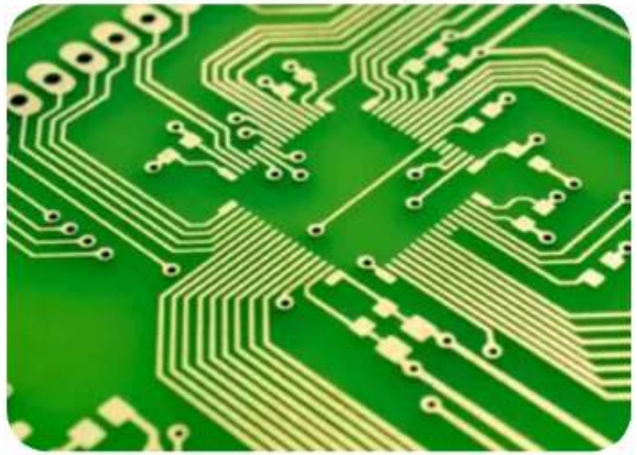
However, its frequency drifts with temperature (±800 Hz at 16 MHz), aging (~5 ppm/year), and mechanical vibrations. The ideal solution would be an oven-controlled oscillator (OCXO), but its cost is too high for our application

o

Heroumiyani

Our approche : The GPSDO

GPS provides a very accurate pulse every second: the 1 PPS signal (±100 ns). → The quartz ensures short-term stability → GPS ensures long-term accuracy → By combining both, we obtain a frequency source that is stable across all time scales

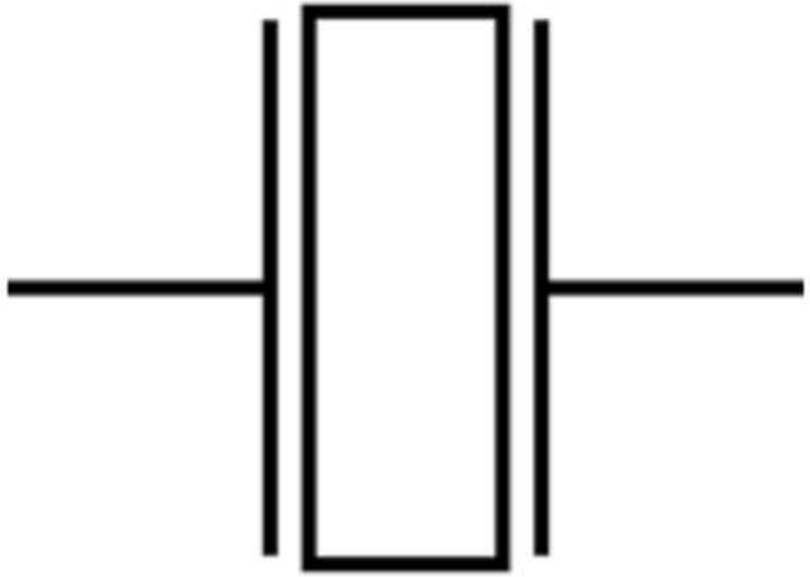


PROJECT OVERVIEW

Content :

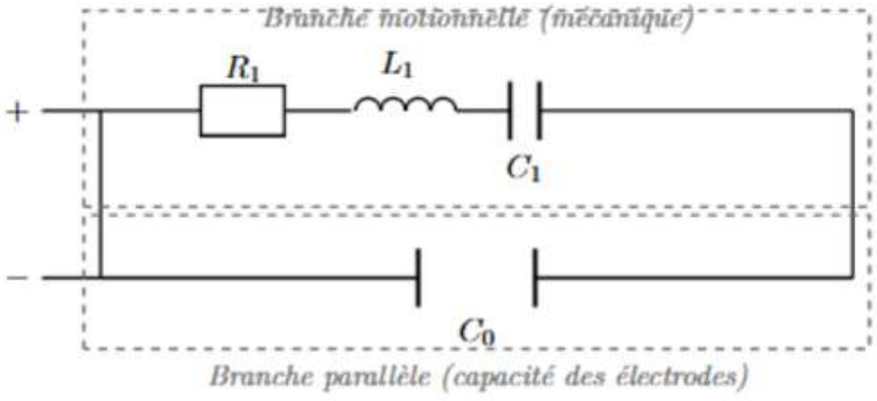
In this project, we will cover many concepts acquired this year: automatic control and PID regulation, analog circuits, radio frequencies, and C programming.

1 — Theoritical Analysis
2 — Schematic Design(KiCAD)
3 — Componenets (BOM)
4 — SPICE Simulation (ngSPICE)
5 — Octave Modelisation
6 — Routing PCB
7 — CMS Soldering
8 — ATmega32U4 Programming



CIRCUIT LOGIC

Varactor Diode

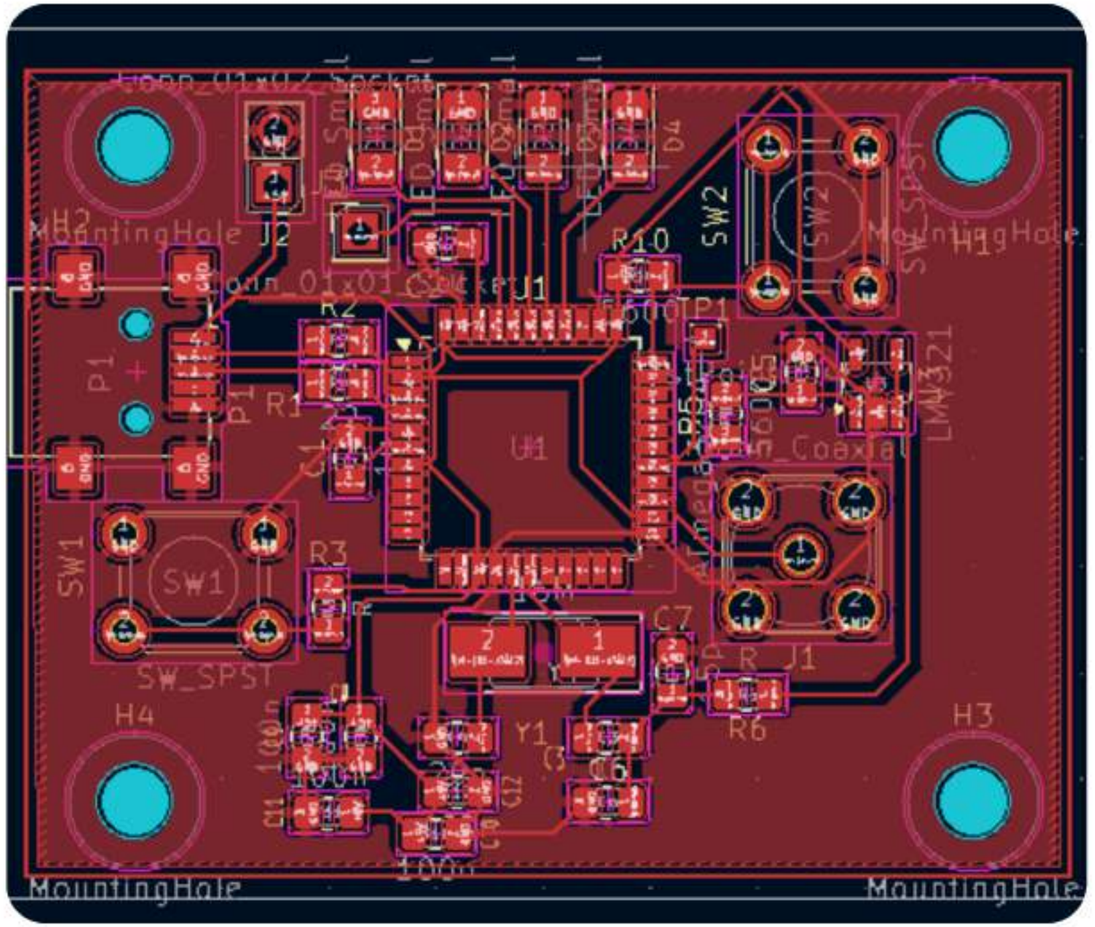


Theoritical Analysis — Howd does it work ?

LThe quartz is electrically modeled by the Butterworth-Van Dyke (BVD) equivalent circuit — it behaves like a highly accurate RLC circuit with a quality factor Q between 30,000 and 60,000.

To correct its frequency, we use a varicap: a diode whose capacitance changes with the applied voltage. capacitance ↑ → frequency ↓

The microcontroller counts the quartz oscillations between two GPS pulses (1 PPS) to measure its exact frequency, then corrects it via a PI controller



SYSTEM ARCHITECTURE & KICAD DESIGN

Diagramme bloc

The microcontroller is the basis of the system :

- Recieves the 1 PPS GPS signal
- Mesure the fréquency over the Input Capture
- Correct via the PI controller
- Commande the varicap via the filtered PWM

KiCAD Schematic

Designed in KiCAD — the schematic includes:

- the RC filter+ buffer
- Decoupling capacitor
- The varicap biasing T-network

PCB Routing & components

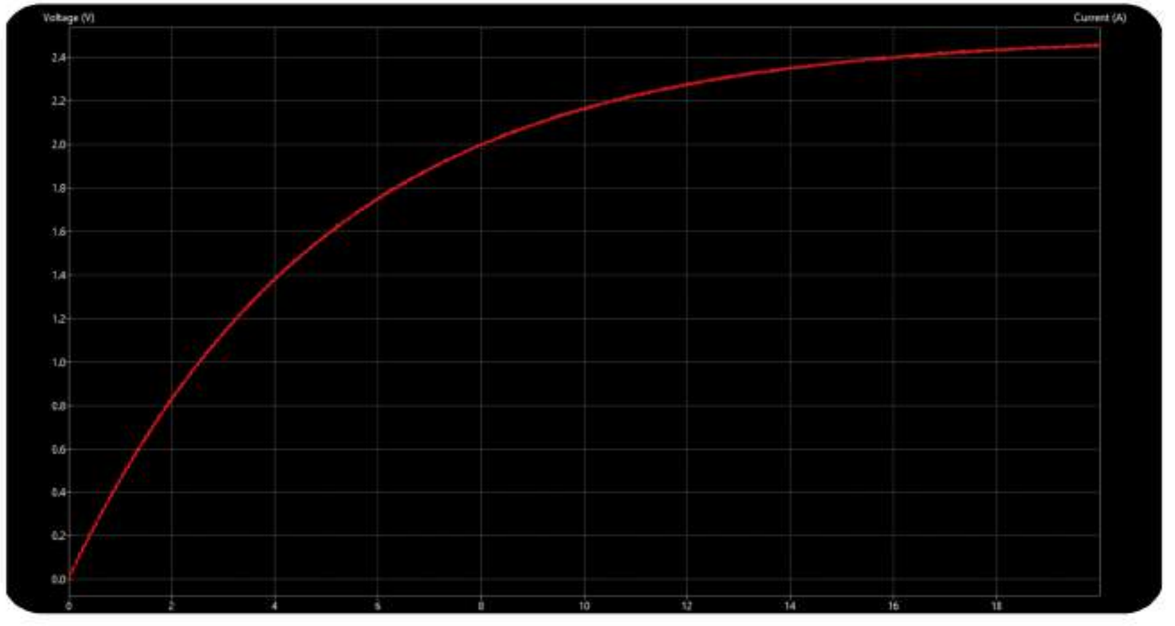
Rules respected during the routing :

- Quartz must be as close as possible to the MCU
- Ground plane on the bottom layer

SIMULATION NGSPICE – FILTRE RC & AMPLIFICATEUR OPÉRATIONNEL

Before manufacturing the circuit, we simulated the behavior of the RC filter and the voltage follower using ngSPICE integrated in KiCAD (AOP file). The circuit includes:

- A VPULSE source simulating the PWM signal
- An RC filter with R = 5.6 kΩ and C = 1 µF
- An op-amp as a voltage follower (gain = +1)



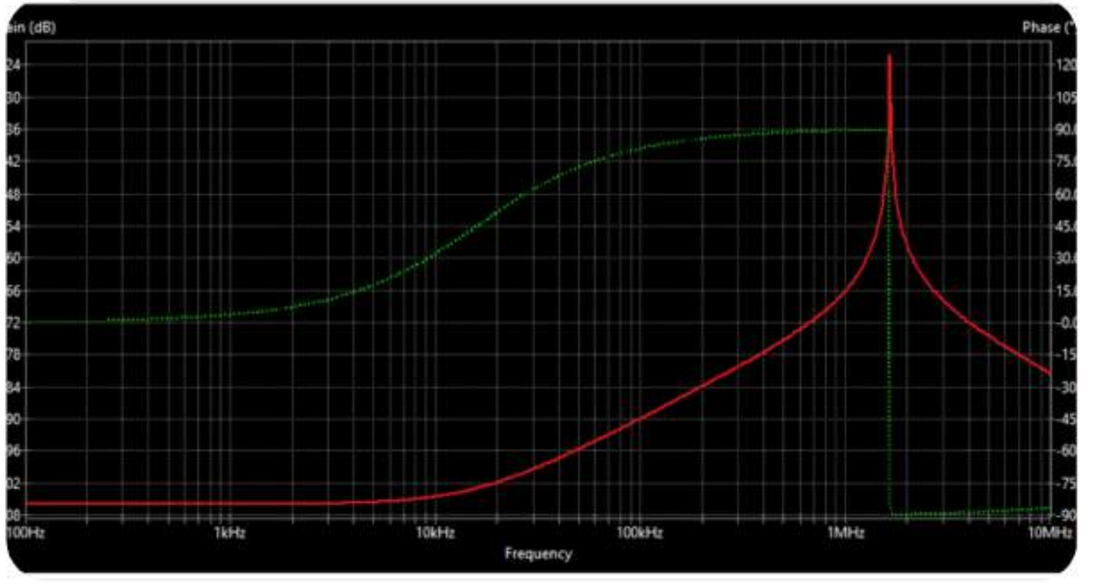
Résults :

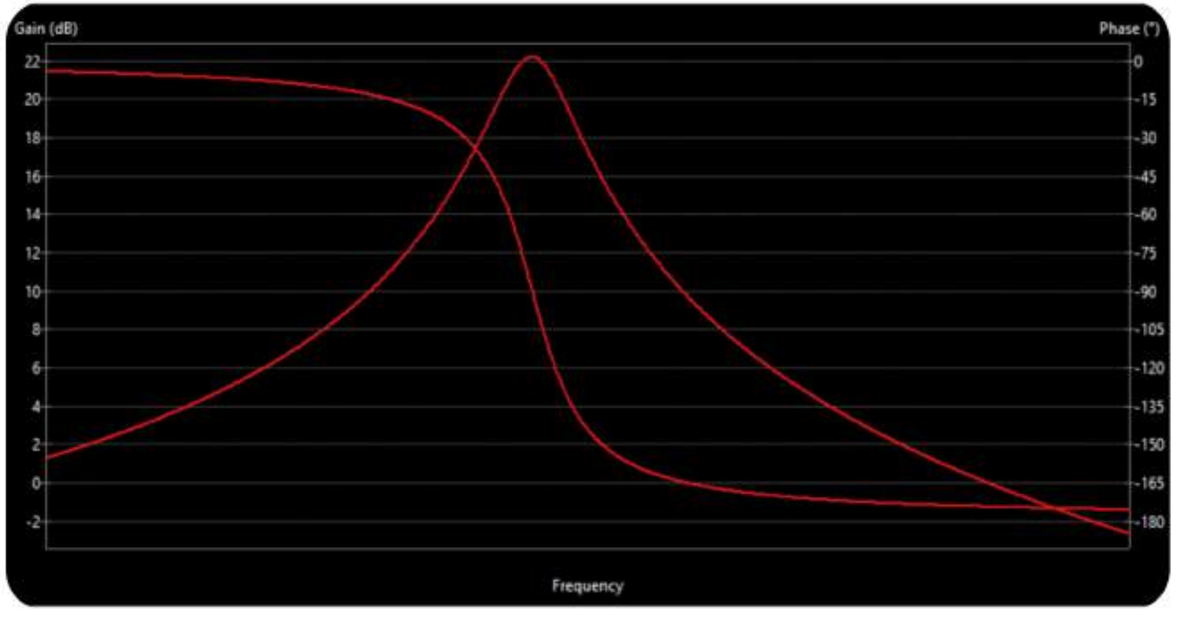
- TRAN simulation → exponential rise, reduced ripple ✓
- AC Sweep → cutoff frequency fc ≈ 28 Hz, -20 dB/decade roll-off ✓

SIMULATION NGSPICE – SIMULATION DU RÉSONATEUR

The quartz is simulated using its Butterworth-Van Dyke model in ngSPICE (resonator circuit file). The circuit includes:

- Motional branch R_1, L_1, C_2 in series
- Static capacitance C_1 in parallel
- Load resistance R_2 at the output"



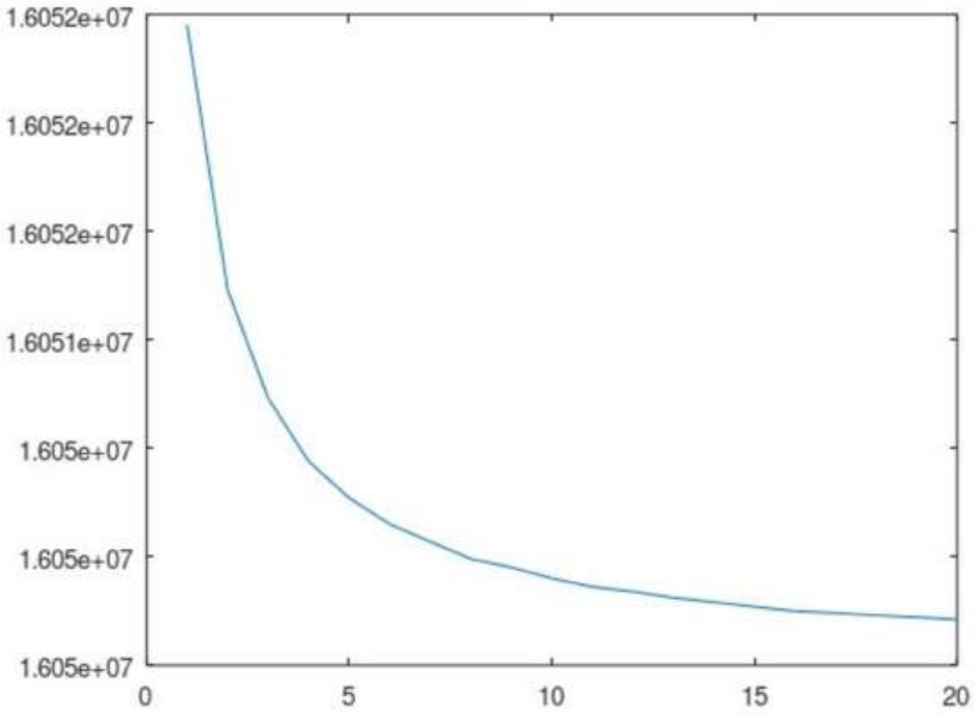


Résults:

- Admittance peak at the series resonance frequency ✓
- Notch at the anti-resonance frequency ✓

"By replacing R_2 with a variable capacitor (resonator circuit capa-legs file), the frequency pulling effect is observed: capacitance ↑ → peak shifts toward lower frequencies ✓"

FREQUENCY PULLING MODELING – OCTAVE

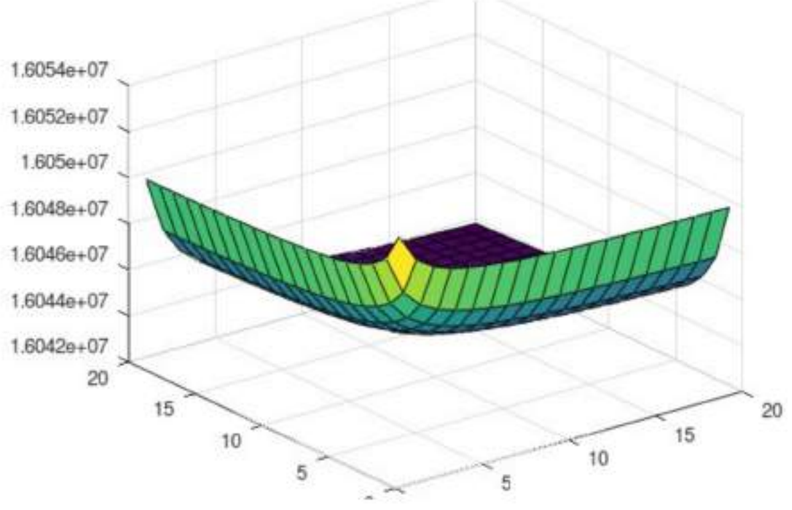


An Octave script drives ngSPICE automatically:

- Varies the capacitances Cp1 and Cp2
- Performs a frequency analysis each time
- Extracts the frequency at the admittance peak
- Stores the results in a solution(m,n,o) matrix

Conclusion :

capacité ↑ → fréquence ↓
Cette propriété confirme que la varicap peut corriger les dérives du quartz de manière précise et contrôlée.



CMS SOLDERING

Soldering order followed:

- USB first → verification of 5V and ground with multimeter
- Passive components → resistors, capacitors
- ATmega32U4
- Through-hole components → buttons, connectors

ATMEGA32U4 & C PROGRAMMING

Why this specific microcontroller ?

- *Native USB integrated*
- *16-bit Timer 1 → Input Capture (1 PPS measurement)*
- *10-bit Timer 4 → PWM (varicap control)*
- *Pierce oscillator → quartz connected directly*

PROGRAMMING – OUR FILES.C

Intro :

The code is organized into several C files, each responsible for a specific task. The whole system runs on the ATmega32U4 and executes in real time, once per second, clocked by the 1 PPS signal. The complete source code is available on our GitHub repository: github.com/Mastene-CHAOUCHE/asservissement-num-rique-d-un-oscillateur-quartz

our progs

inputcap.c — Frequency mesurements

Idea: count the quartz cycles between 2 rising edges of 1 PPS

total = overflow_count * 65536 + (cap2 - cap1);

PWM

Idea: simulate a DAC using a 10-bit PWM signal

OCR4D = value; // 0 to 1023 → 0V to 5V

PID.c — PI Controller

Idea: correct the frequency error every second

command = Kp * error + Ki * integral;

CONCLUSION

ATCE PROJECT

This project allowed us to design a complete GPSDO system, from theoretical analysis to microcontroller programming.

Key Takeways :

Quartz is stable in the short term but drifts → GPS corrects it in the long term SPICE simulation validates the circuit before manufacturing The varicap enables fine and controlled frequency adjustment The PI controller maintains the frequency within ±2 Hz of the setpoint SMD soldering requires method and patience

Going Further :

Use an oven-controlled oscillator (OCXO) Implement Allan variance to monitor the quality of the frequency source

THANK YOU FOR YOUR ATTENTION!

We would like to warmly thank our supervisors Prof. Jean-Michel Friedt and Prof. Emile Carry for their availability, guidance, and support throughout this project. All project resources are available on our GitHub repository: KiCAD schematics, simulation files, C source code, BOM and PCB. github.com/Mastene-CHAOUCHE/asservissement-num-rique-d-un-oscillateur-quartz

Do you have any questions? Feel free to ask, we are here to answer them!

UNIVERSITE MARIE & LOUIS PASTEUR

ASSERVISSEMENT NUMERIQUE D'UN OSCILLATEUR A QUARTZ

PROJET DE ATCE

presente par :

HEROUMI Yani
NEGRECHE Aaron
ATMANI Aymene
CHAOUCHE Mastene
BELAIDI Katia
ATAMNIA Mondher

encadre par :
Pr. Jean Michel Friedt
Pr. Emile Carry

Année universitaire : 2025/2026

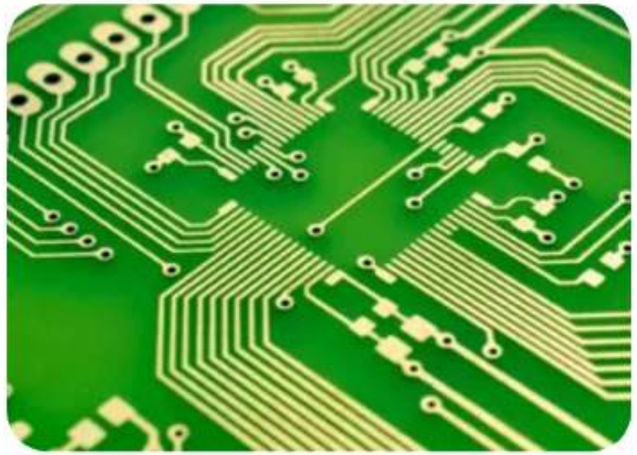
POURQUOI A-T-ON BESOIN D'UN GPSDO ?

Le quartz est très utilisé en électronique car il est précis, petit et peu coûteux.

mais sa fréquence dérive avec la température (±800 Hz à 16 MHz), le vieillissement (~5 ppm/an) et les vibrations mécaniques.
La solution idéale serait un oscillateur thermostaté (OCXO), mais son coût est trop élevé pour notre application

Notre approche : le GPSDO

Le GPS fournit une impulsion très précise chaque seconde : le signal 1 PPS (±100 ns).
→ Le quartz assure la stabilité court terme → Le GPS assure la précision long terme → En combinant les deux, on obtient une source de fréquence stable sur toutes les échelles de temps

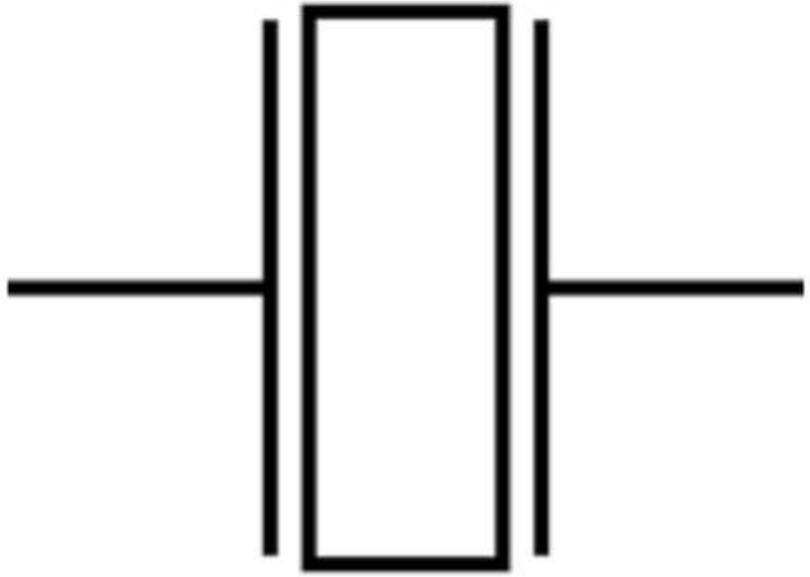


DÉROULEMENT DU PROJET

Contenu :

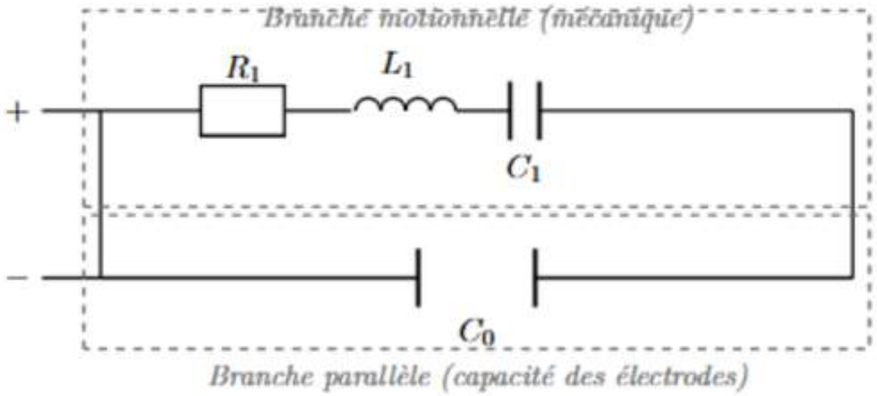
Dans ce projet, nous allons aborder de nombreux concepts acquis cette année : automatique et régulation PID, circuits analogiques, radiofréquences et programmation en C.

1 — Analyse théorique
2 — Conception du schéma (KiCAD)
3 — Choix des composants (BOM)
4 — Simulation SPICE (ngSPICE)
5 — Modélisation Octave
6 — Routage PCB
7 — Soudure CMS
8 — Programmation ATmega32U4



LOGIQUE DU CIRCUIT

Varactor Diode



Analyse théorique — Comment ça marche ?

Le quartz est modélisé électriquement par le schéma de Butterworth-Van Dyke (BVD) — il se comporte comme un circuit RLC très précis avec un facteur de qualité Q entre 30 000 et 60 000.
Pour corriger sa fréquence on utilise une varicap : une diode dont la capacité change avec la tension appliquée.
capacité ↑ → fréquence ↓

Le microcontrôleur compte les oscillations du quartz entre deux impulsions GPS (1 PPS) pour mesurer sa fréquence exacte, puis corrige via un régulateur PI.

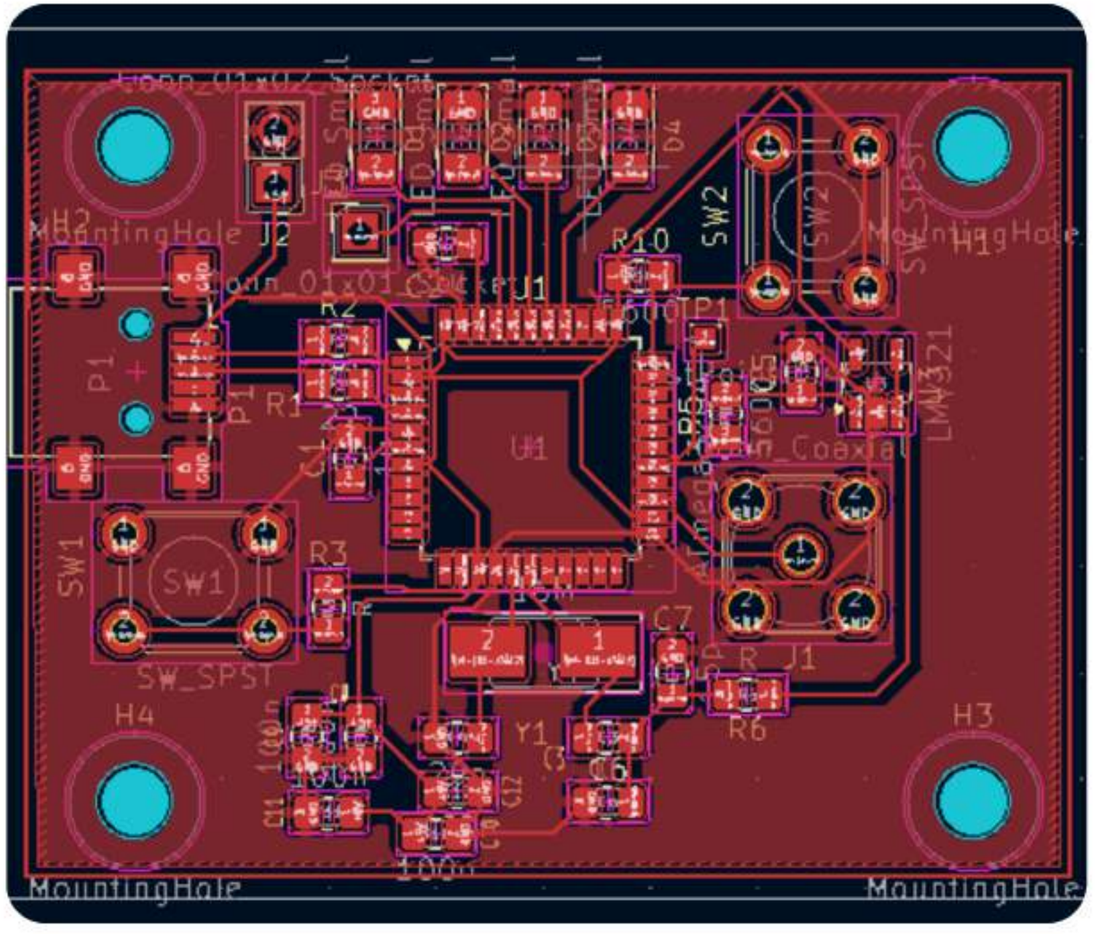


Diagramme bloc

Le microcontrôleur est au cœur du système :

- Reçoit le 1 PPS du GPS
- Mesure la fréquence par Input Capture
- Corrige via le régulateur PI
- Commande la varicap via PWM filtré

Schéma électronique KiCAD

Conçu sous KiCAD - le schéma regroupe :

- Le filtre RC + suiveur
- Les condensateurs de découplage
- Le T de polarisation de la varicap

Routage PCB & composants

Règles respectées lors du routage :

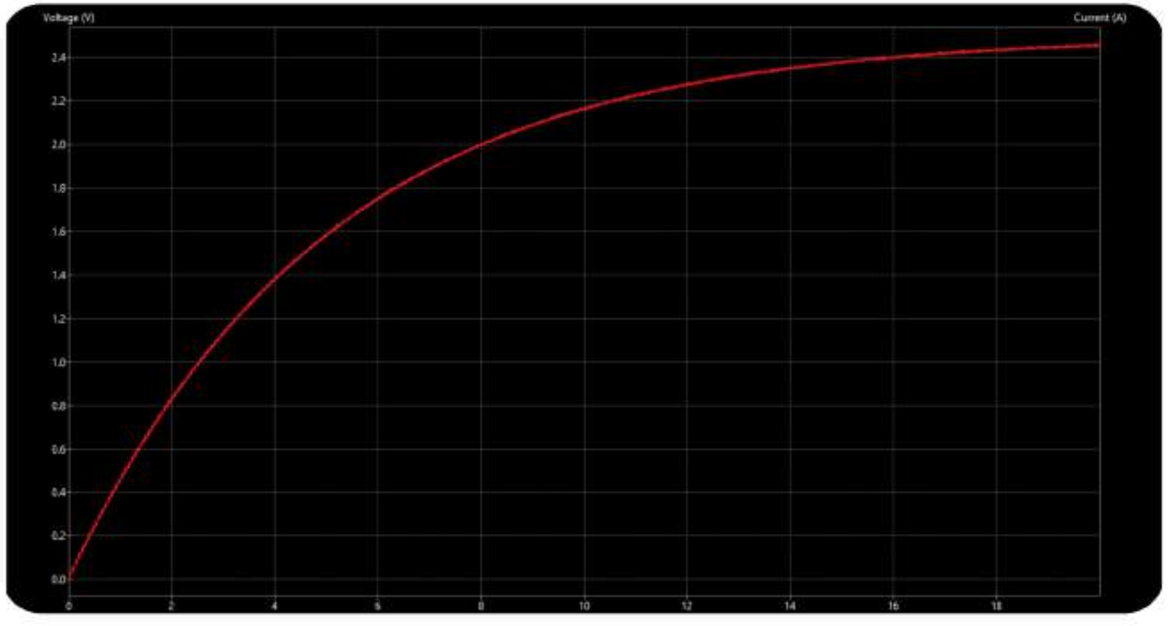
- Quartz au plus près du MCU
- Plan de masse en couche inférieure

ARCHITECTURE DU SYSTÈME & CONCEPTION KICAD

SIMULATION NGSPICE – FILTRE RC & AMPLIFICATEUR OPÉRATIONNEL

Avant de fabriquer le circuit nous avons simulé le comportement du filtre RC et du suiveur sous ngSPICE intégré dans KiCAD (fichier AOP).
Le circuit comprend :

- Une source VPULSE simulant le signal PWM
- Un filtre RC avec R = 5,6 kΩ et C = 1 µF
- Un AOP en suiveur (gain = +1)



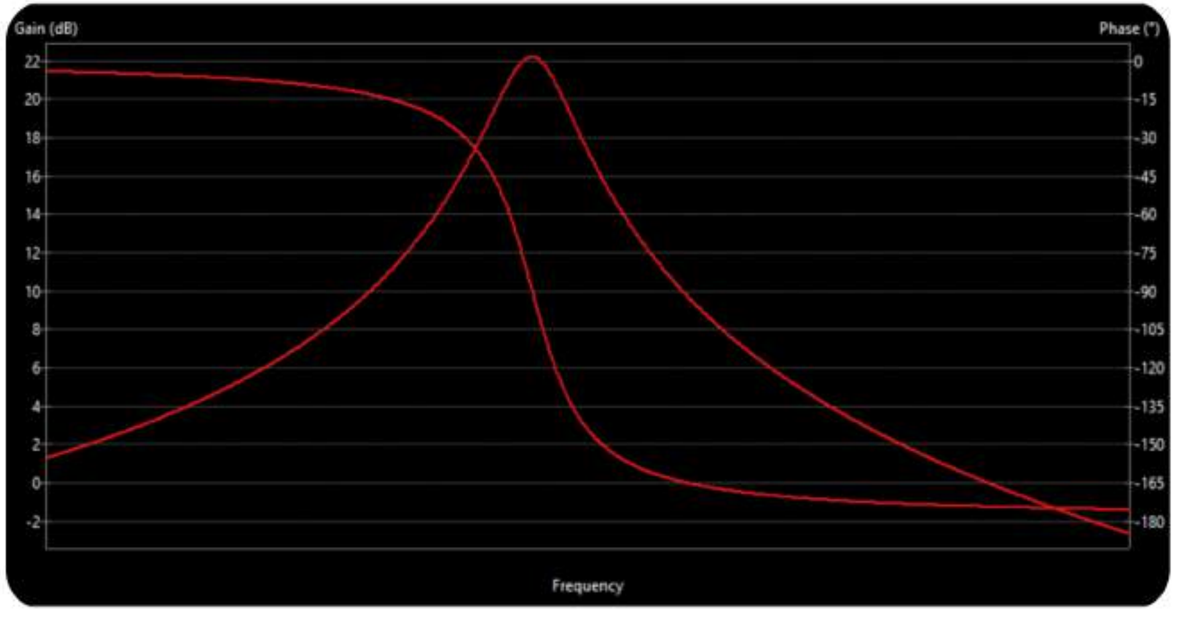
Résultats :

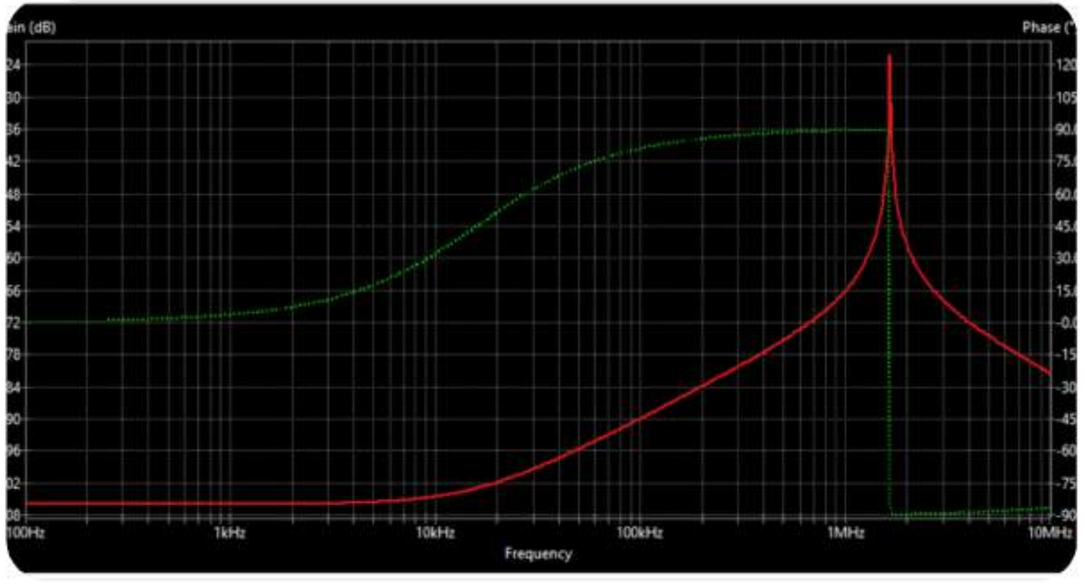
- Simulation TRAN → montée exponentielle, ondulations réduites ✓
- AC Sweep → fréquence de coupure fc ≈ 28 Hz, décroissance -20 dB/décade ✓

SIMULATION NGSPICE – SIMULATION DU RÉSONATEUR

Le quartz est simulé via son modèle Butterworth-Van Dyke sous ngSPICE (fichier resonator circuit).
Le circuit comprend :

- Branche motionnelle R_1, L_1, C_2 en série
- Capacité statique C_1 en parallèle
- Résistance de charge R_2 en sortie





Résultats :

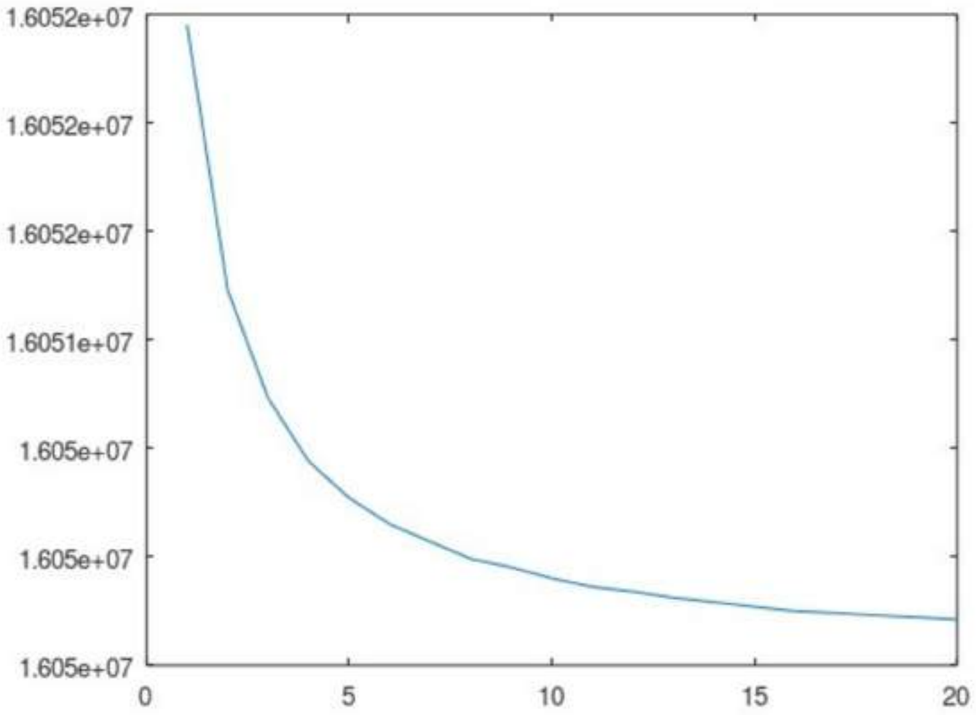
- Pic d'admittance à la fréquence de résonance série ✓
- Creux à la fréquence d'antirésonance ✓

En remplaçant R_2 par un condensateur variable (fichier resonator circuit capa-legs) on observe le tirage en fréquence :
capacité ↑ → pic se déplace vers les basses fréquences ✓

MODÉLISATION DU TIRAGE EN FRÉQUENCE – OCTAVE

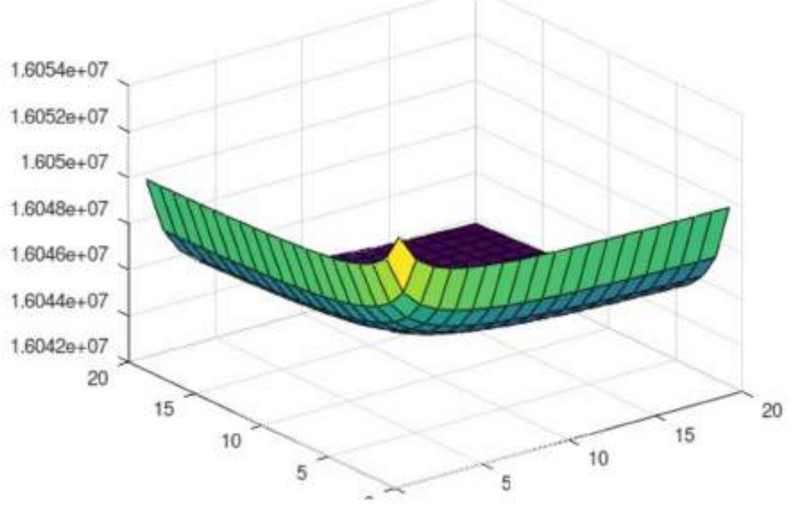
Un script Octave pilote ngSPICE automatiquement :

- Fait varier les capacités Cp1 et Cp2
- Effectue une analyse fréquentielle à chaque fois
- Extrait la fréquence au maximum d'admittance
- Stocke les résultats dans une matrice solution(m,n,o)



Conclusion :

capacité ↑ → fréquence ↓
Cette propriété confirme que la varicap peut corriger les dérives du quartz de manière précise et contrôlée.



SOUDURE CMS

Ordre de soudure respecté :

- USB en premier → vérification 5V et masse au multimètre
- Composants passifs → résistances, condensateurs
- ATmega32U4
- Composants traversants → boutons, connecteurs

ATMEGA32U4 & PROGRAMMATION C

Pourquoi ce microcontrôleur ?

- *USB natif intégré*
- *Timer 1 16 bits → Input Capture (mesure 1 PPS)*
- *Timer 4 10 bits → PWM (commande varicap)*
- *Oscillateur Pierce → quartz connecté directement*

PROGRAMMATION – NOS FICHIERS.C

Intro :

Le code est organisé en plusieurs fichiers C, chacun responsable d'une tâche précise. L'ensemble tourne sur l'ATmega32U4 et s'exécute en temps réel, une fois par seconde cadencé par le signal 1 PPS. L'ensemble du code source est disponible sur notre dépôt GitHub : github.com/Mastene-CHAOUCHE/asservissement-num-rique-d-un-oscillateur-quartz

NOS PROGS

inputcap.c — Mesure de fréquence

Idée : compter les cycles du quartz entre 2 fronts 1 PPS

total = overflow_count * 65536 + (cap2 - cap1);

PWM

Idée : simuler un DAC avec un signal PWM 10 bits

OCR4D = valeur; // 0 à 1023 → 0V à 5V

PID.c — Correcteur PI

Idée : corriger l'erreur de fréquence à chaque seconde

commande = Kp * erreur + Ki * integrale;

CONCLUSION
PROJET DE ATCE

Ce projet nous a permis de concevoir un système GPSDO complet, de l'analyse théorique jusqu'à la programmation du microcontrôleur.

Ce que nous avons retenu :

Le quartz est stable à court terme mais dérive → le GPS corrige à long terme

La simulation SPICE valide le circuit avant fabrication

La varicap permet un réglage fin et contrôlé de la fréquence

Le régulateur PI maintient la fréquence à ±2 Hz de la consigne

La soudure CMS demande méthode et patience

Pour aller plus loin :

Utiliser un quartz thermostaté (OCXO)

Implémenter la variance d'Allan pour monitorer la qualité de la source

THANK YOU FOR YOUR ATTENTION!

Nous tenons à remercier chaleureusement nos encadrants Pr. Jean-Michel Friedt et Pr. Emile Carry pour leur disponibilité, leurs conseils et leur accompagnement tout au long de ce projet.

Toutes les ressources du projet sont disponibles sur notre dépôt GitHub : schémas KiCAD, fichiers de simulation, code source C, BOM et PCB.

github.com/Mastene-CHAOUCHE/asservissement-num-rique-d-un-oscillateur-quartz

Avez-vous des questions ? N'hésitez pas, nous sommes là pour y répondre !